



university of  
 groningen

# Models and Semantics of Computation (MaSC)

## Lecture 10: Behavioral Equivalence in CCS

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI  
University of Groningen, Groningen, the Netherlands



$$p \xrightarrow{\mu} q$$

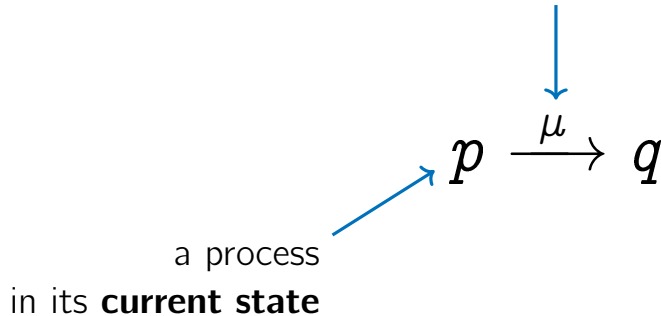


$$p \xrightarrow{\mu} q$$

a process  
in its **current state**

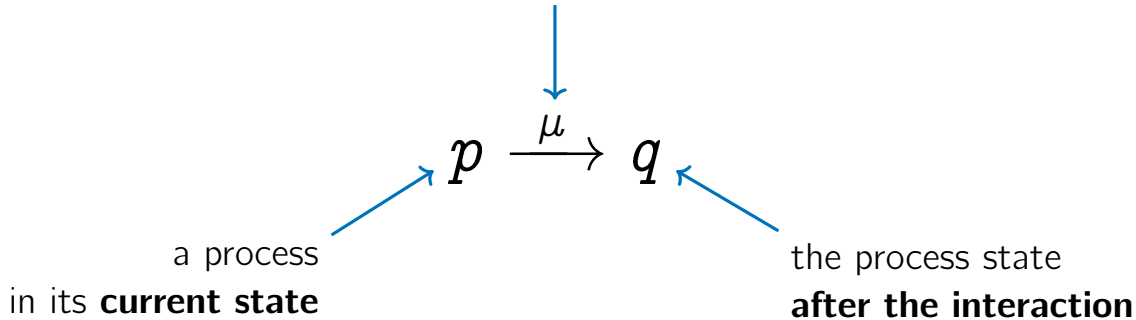


**ongoing interaction** with  
the environment (other processes)





**ongoing interaction** with  
the environment (other processes)





|                         |                                  |
|-------------------------|----------------------------------|
| $p, q ::= \text{nil}$   | inactive process                 |
| $  x$                   | process variable (for recursion) |
| $  \mu.p$               | action prefix                    |
| $  p \backslash \alpha$ | restricted channel               |
| $  p[\phi]$             | channel relabeling               |
| $  p + q$               | non deterministic choice         |
| $  p \mid q$            | parallel composition             |
| $  \text{rec } x. p$    | recursion                        |

(Note: operators are listed in order of precedence)

# Structural Operational Semantics



$$\begin{array}{c}
 \text{act} \\
 \frac{}{\mu.p \xrightarrow{\mu} p}
 \end{array}
 \quad
 \begin{array}{c}
 \text{SumL} \\
 \frac{p_1 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}
 \end{array}
 \quad
 \begin{array}{c}
 \text{SumR} \\
 \frac{p_2 \xrightarrow{\mu} q}{p_1 + p_2 \xrightarrow{\mu} q}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Rec} \\
 \frac{p[\text{rec } x. p/x] \xrightarrow{\mu} q}{\text{rec } x. p \xrightarrow{\mu} q}
 \end{array}$$
  

$$\begin{array}{c}
 \text{ParL} \\
 \frac{p_1 \xrightarrow{\mu} q_1}{p_1 \mid p_2 \xrightarrow{\mu} q_1 \mid p_2}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Com} \\
 \frac{p_1 \xrightarrow{\lambda} q_1 \quad p_2 \xrightarrow{\bar{\lambda}} q_2}{p_1 \mid p_2 \xrightarrow{\tau} q_1 \mid q_2}
 \end{array}
 \quad
 \begin{array}{c}
 \text{ParR} \\
 \frac{p_2 \xrightarrow{\mu} q_2}{p_1 \mid p_2 \xrightarrow{\mu} p_1 \mid q_2}
 \end{array}$$
  

$$\begin{array}{c}
 \text{Res} \\
 \frac{p \xrightarrow{\mu} q \quad \mu \notin \{\alpha, \bar{\alpha}\}}{p \backslash \alpha \xrightarrow{\mu} q \backslash \alpha}
 \end{array}
 \quad
 \begin{array}{c}
 \text{Rel} \\
 \frac{p \xrightarrow{\mu} q}{p[\phi] \xrightarrow{\phi(\mu)} q[\phi]}
 \end{array}$$

# Linked Buffers (1)



$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array} \quad \begin{array}{c} B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\ \xleftarrow{\bar{c}} \end{array} \quad \begin{array}{c} B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\ \xleftarrow{\overline{out}} \end{array}$$

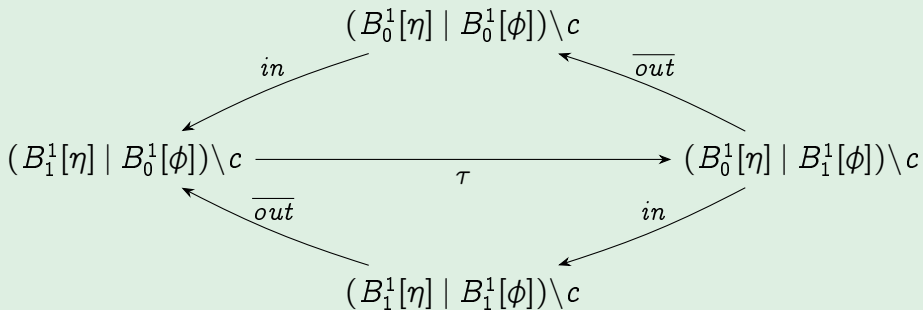
Consider the linked system  $(B_0^1[\eta] \mid B_1^1[\phi]) \setminus c$

# Linked Buffers (1)



$$\begin{array}{ll}
 B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\
 B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c
 \end{array}
 \quad
 \begin{array}{ccc}
 B_0^1[\eta] & \xrightleftharpoons[in]{in} & B_1^1[\eta] \\
 B_0^1[\phi] & \xrightleftharpoons[out]{c} & B_1^1[\phi]
 \end{array}$$

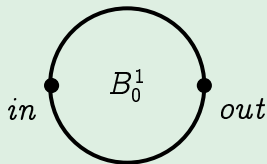
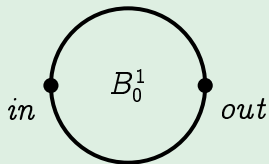
Consider the linked system  $(B_0^1[\eta] \mid B_0^1[\phi]) \setminus c$



## Linked Buffers (2)



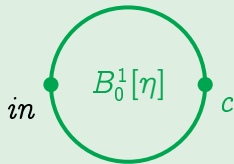
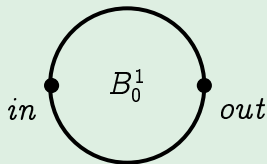
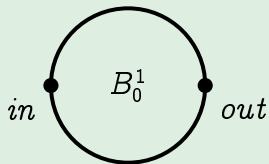
$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array}$$
$$B_0^1[\eta] \begin{array}{c} \xrightarrow{in} \\ \xleftarrow{\bar{c}} \end{array} B_1^1[\eta] \qquad B_0^1[\phi] \begin{array}{c} \xrightarrow{c} \\ \xleftarrow{\overline{out}} \end{array} B_1^1[\phi]$$



## Linked Buffers (2)



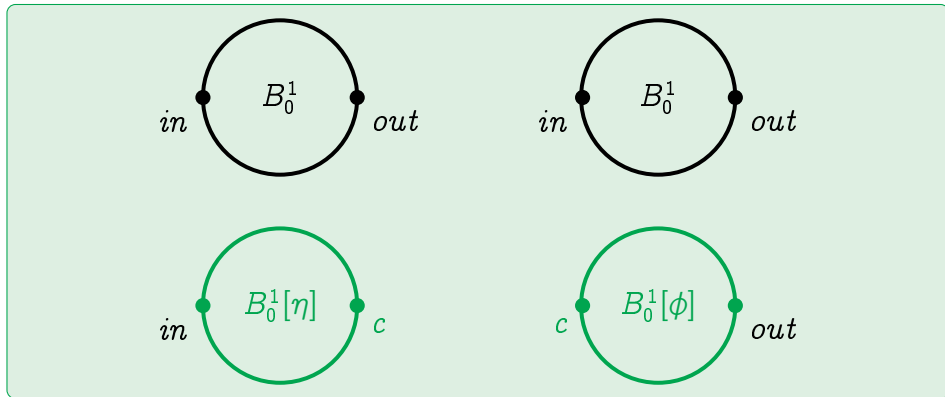
$$\begin{array}{ll} B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\ B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c \end{array} \quad \begin{array}{c} B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\ \xleftarrow{\bar{c}} \end{array} \quad \begin{array}{c} B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\ \xleftarrow{\overline{out}} \end{array}$$



# Linked Buffers (2)



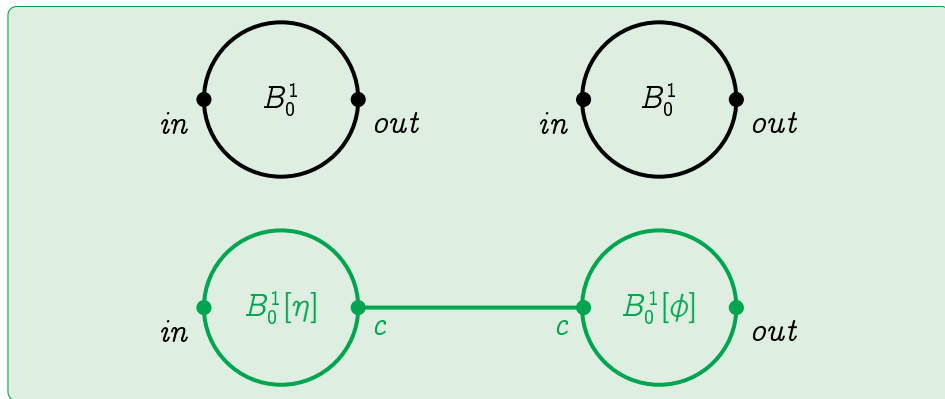
$$\begin{array}{ll}
 B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\
 B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c
 \end{array}
 \quad
 \begin{array}{ccc}
 B_0^1[\eta] & \xrightleftharpoons[in]{in} & B_1^1[\eta] \\
 B_0^1[\phi] & \xrightleftharpoons[out]{c} & B_1^1[\phi]
 \end{array}$$



# Linked Buffers (2)



$$\begin{array}{ll}
 B_0^1 \triangleq in.B_1^1 & \eta(out) = c \\
 B_1^1 \triangleq \overline{out}.B_0^1 & \phi(in) = c
 \end{array}
 \quad
 \begin{array}{c}
 B_0^1[\eta] \xrightarrow{in} B_1^1[\eta] \\
 \xleftarrow{\bar{c}}
 \end{array}
 \quad
 \begin{array}{c}
 B_0^1[\phi] \xrightarrow{c} B_1^1[\phi] \\
 \xleftarrow{\overline{out}}
 \end{array}$$



# Linked Buffers (3)



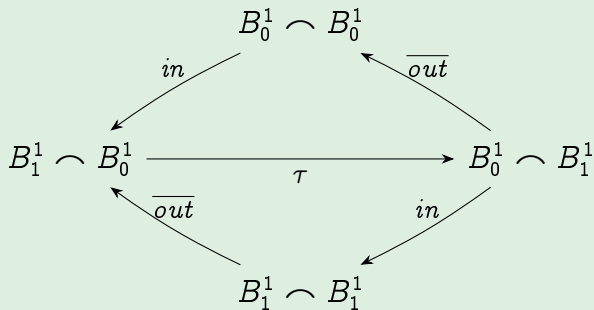
$$B_0^1 \triangleq in.B_1^1$$

$$\eta(out) = c$$

$$p \curvearrowright q \triangleq (p[\eta] \mid q[\phi]) \setminus c$$

$$B_1^1 \triangleq \overline{out}.B_0^1$$

$$\phi(in) = c$$



# Linked Boolean Buffers



$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$B_{tt} \triangleq \overline{out}_{tt}.B_{\emptyset}$$

$$B_{ff} \triangleq \overline{out}_{ff}.B_{\emptyset}$$

$$\eta(out_{ff}) = c_{ff}$$

$$\phi(in_{ff}) = c_{ff}$$

$$\eta(out_{tt}) = c_{tt}$$

$$\phi(in_{tt}) = c_{tt}$$

# Linked Boolean Buffers



$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$B_{tt} \triangleq \overline{out_{tt}}.B_{\emptyset}$$

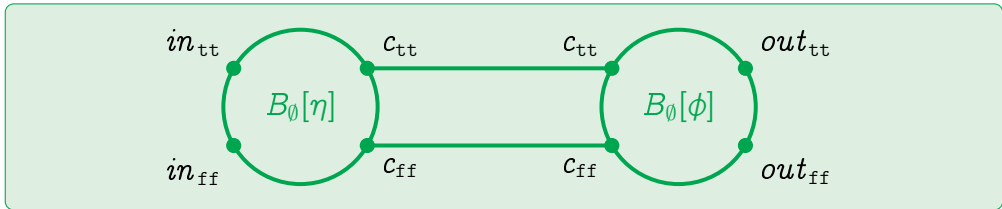
$$B_{ff} \triangleq \overline{out_{ff}}.B_{\emptyset}$$

$$\eta(out_{ff}) = c_{ff}$$

$$\eta(out_{tt}) = c_{tt}$$

$$\phi(in_{ff}) = c_{ff}$$

$$\phi(in_{tt}) = c_{tt}$$



# Linked Boolean Buffers: Exercise



$$B_{\emptyset} \triangleq in_{tt}.B_{tt} + in_{ff}.B_{ff}$$

$$\eta(out_{ff}) = c_{ff}$$

$$\eta(out_{tt}) = c_{tt}$$

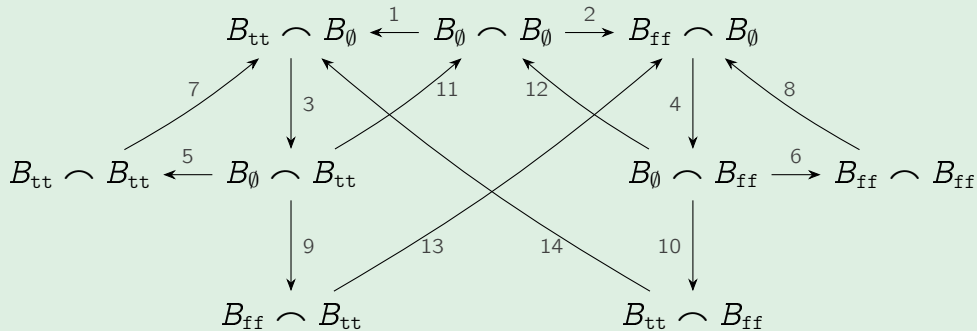
$$B_{tt} \triangleq \overline{out}_{tt}.B_{\emptyset}$$

$$\phi(in_{ff}) = c_{ff}$$

$$\phi(in_{tt}) = c_{tt}$$

$$B_{ff} \triangleq \overline{out}_{ff}.B_{\emptyset}$$

$$p \curvearrowright q \triangleq (p[\eta] \mid q[\phi]) \setminus \{c_{ff}, c_{tt}\}$$



- ▶ A web-based tool for specifying and verifying processes expressed in CCS
- ▶ In particular, you can explore the LTS of the given processes
- ▶ Good didactic approach to understand the semantics of processes
- ▶ URL: <https://caal.cs.aau.dk/>

# CCS with Value Passing



- We have seen actions  $\alpha$  and  $\bar{\alpha}$ . A convenient extension, in which  $\alpha$  acts as a channel for sending and receiving a value:

$\text{send}_v$

$\text{recv}_v$

$$\frac{}{\alpha!v.p \xrightarrow{\bar{\alpha}_v} p}$$

$$\frac{}{\alpha?x.p \xrightarrow{\alpha_v} p[v/x]}$$

# CCS with Value Passing



- We have seen actions  $\alpha$  and  $\bar{\alpha}$ . A convenient extension, in which  $\alpha$  acts as a channel for sending and receiving a value:

$$\begin{array}{c} \text{send}_v \\ \hline \alpha!v.p \xrightarrow{\bar{\alpha}_v} p \end{array} \qquad \begin{array}{c} \text{recv}_v \\ \hline \alpha?x.p \xrightarrow{\alpha_v} p[v/x] \end{array}$$

- Assuming a finite set of values  $V \triangleq \{v_1, \dots, v_n\}$ , the value-passing constructs are **representable** in plain CCS.
- Formally, we define a **translation**. Main cases:

$$\llbracket \alpha!v_i.p \rrbracket = \bar{\alpha}_{v_i}.\llbracket p \rrbracket$$

$$\llbracket \alpha?x.p \rrbracket = \alpha_{v_1}.\llbracket p[v_1/x] \rrbracket + \dots + \alpha_{v_n}.\llbracket p[v_n/x] \rrbracket$$

# CCS with Value Passing



- ▶ We have seen actions  $\alpha$  and  $\bar{\alpha}$ . A convenient extension, in which  $\alpha$  acts as a channel for sending and receiving a value:

$$\begin{array}{c} \text{send}_v \\ \hline \alpha!v.p \xrightarrow{\bar{\alpha}_v} p \end{array} \qquad \begin{array}{c} \text{recv}_v \\ \hline \alpha?x.p \xrightarrow{\alpha_v} p[v/x] \end{array}$$

- ▶ Assuming a finite set of values  $V \triangleq \{v_1, \dots, v_n\}$ , the value-passing constructs are **representable** in plain CCS.
- ▶ Formally, we define a **translation**. Main cases:

$$\begin{aligned} \llbracket \alpha!v_i.p \rrbracket &= \bar{\alpha}_{v_i}.\llbracket p \rrbracket \\ \llbracket \alpha?x.p \rrbracket &= \alpha_{v_1}.\llbracket p[v_1/x] \rrbracket + \dots + \alpha_{v_n}.\llbracket p[v_n/x] \rrbracket \end{aligned}$$

- ▶ Hence, with finite  $V$ , value passing is a **syntactic sugar**.

## Exercise: Find the LTS (1)



$$P \triangleq \text{coin}.\overline{\text{coffee}} + \text{coin}.\overline{\text{tea}}$$

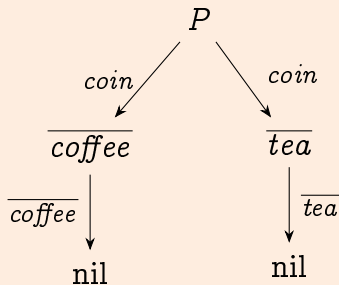
$$Q \triangleq \text{coin}.\overline{(\text{coffee} + \text{tea})}$$

## Exercise: Find the LTS (1)



$$P \triangleq \text{coin}.\overline{\text{coffee}} + \text{coin}.\overline{\text{tea}}$$

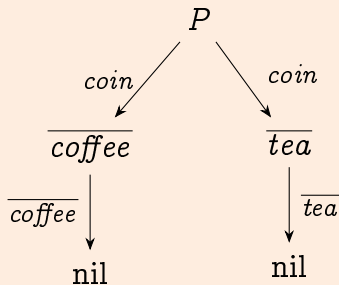
$$Q \triangleq \text{coin}.\overline{(\text{coffee} + \text{tea})}$$



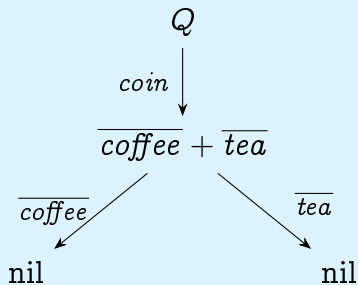
## Exercise: Find the LTS (1)



$$P \triangleq \text{coin}.\overline{\text{coffee}} + \text{coin}.\overline{\text{tea}}$$



$$Q \triangleq \text{coin}.\overline{(\text{coffee} + \text{tea})}$$



## Exercise: Find the LTS (2)



$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$

$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$

$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$

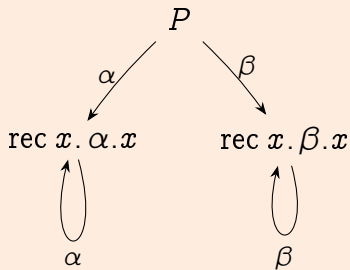
## Exercise: Find the LTS (2)



$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$

$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$

$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$



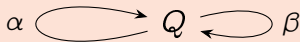
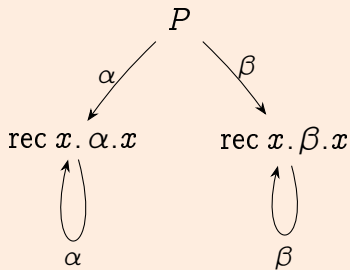
## Exercise: Find the LTS (2)



$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$

$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$

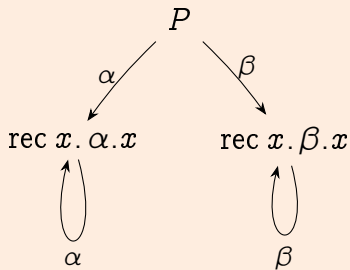
$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$



## Exercise: Find the LTS (2)



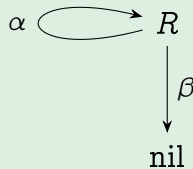
$$P \triangleq \text{rec } x. \alpha.x + \text{rec } x. \beta.x$$



$$Q \triangleq \text{rec } x. (\alpha.x + \beta.x)$$



$$R \triangleq \text{rec } x. (\alpha.x + \beta.\text{nil})$$



## Exercise: Find the LTS (3)



$$T \triangleq \text{rec } x. \alpha. (\text{nil} \mid x) + \beta. \text{nil}$$

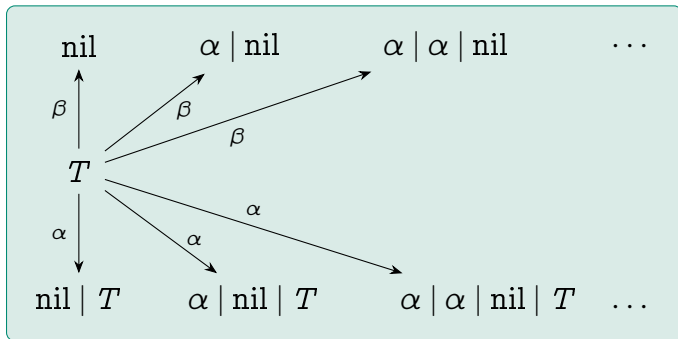
$$T \triangleq (\alpha \mid T) + \beta$$

## Exercise: Find the LTS (3)



$$T \triangleq \text{rec } x. \alpha.(\text{nil} \mid x) + \beta.\text{nil}$$

$$T \triangleq (\alpha \mid T) + \beta$$

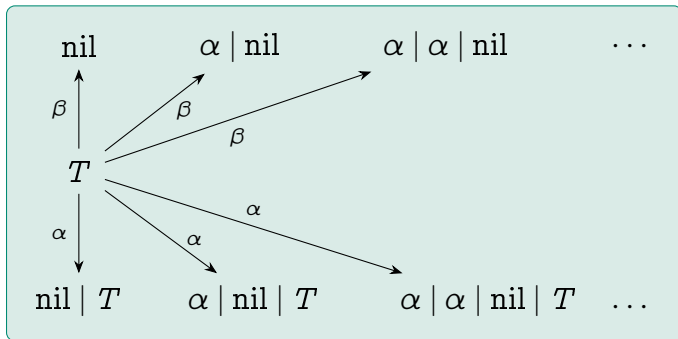


## Exercise: Find the LTS (3)



$$T \triangleq \text{rec } x. \alpha.(\text{nil} \mid x) + \beta.\text{nil}$$

$$T \triangleq (\alpha \mid T) + \beta$$



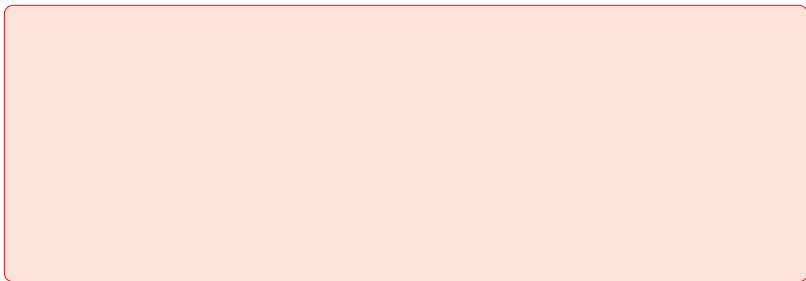
Suggestion: Try running this process in CAAL!

## Exercise: Find the LTS (4)



$$U \triangleq \text{rec } x. \alpha \mid \beta.x$$

$$U \triangleq \alpha \mid (\beta.U)$$

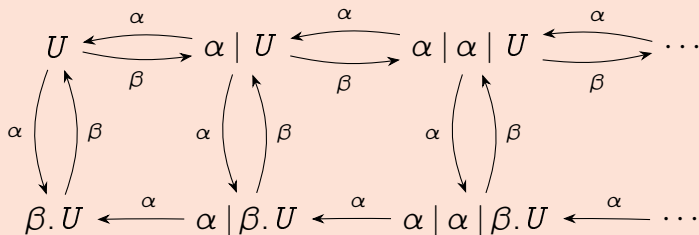


# Exercise: Find the LTS (4)



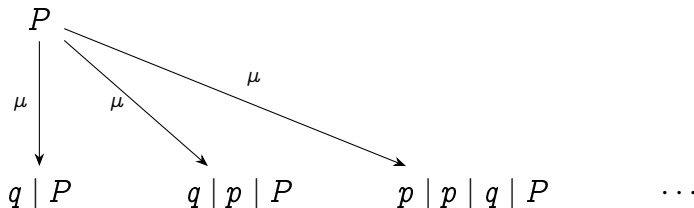
$$U \triangleq \text{rec } x. \alpha \mid \beta.x$$

$$U \triangleq \alpha \mid (\beta.U)$$





- ▶ As we have seen, CCS allows a very general form of recursion  
There are processes with infinitely many outgoing transitions
- ▶ Suppose  $p \xrightarrow{\mu} q$  and take  $P \triangleq \text{rec } x. p \mid x$  (that is:  $P \triangleq p \mid P$ ).  
We have:



- ▶ We would like to avoid this kind of **infinitely branching** processes.

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

► `rec  $x$ .  $x$`

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

► `rec  $x$ .  $x$`

**X**

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

►  $\text{rec } x. x$

**X**

►  $\text{rec } x. \alpha. \text{rec } y. x$

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$  **X**
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  **✓**

# Guarded Processes



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$  ✗
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  ✓
- ▶  $\text{rec } x. \alpha. \text{rec } y. x + y$



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- |                                                 |          |
|-------------------------------------------------|----------|
| ▶ $\text{rec } x. x$                            | $\times$ |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x$     | ✓        |
| ▶ $\text{rec } x. \alpha. \text{rec } y. x + y$ | $\times$ |



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$   $\times$
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  ✓
- ▶  $\text{rec } x. \alpha. \text{rec } y. x + y$   $\times$
- ▶  $\text{rec } x. \alpha. \text{rec } y. x \mid y$



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- |                                                    |          |
|----------------------------------------------------|----------|
| ► $\text{rec } x. x$                               | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x$        | ✓        |
| ► $\text{rec } x. \alpha. \text{rec } y. x + y$    | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x \mid y$ | $\times$ |



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- ▶  $\text{rec } x. x$  ✗
- ▶  $\text{rec } x. \alpha. \text{rec } y. x$  ✓
- ▶  $\text{rec } x. \alpha. \text{rec } y. x + y$  ✗
- ▶  $\text{rec } x. \alpha. \text{rec } y. x \mid y$  ✗
- ▶  $\text{rec } x. \alpha. \text{rec } y. x \mid \beta. y$



Processes in which process variables occur under a prefix. That is, recursion occurs **guarded** by some action.

Guarded?

- |                                                           |          |
|-----------------------------------------------------------|----------|
| ► $\text{rec } x. x$                                      | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x$               | ✓        |
| ► $\text{rec } x. \alpha. \text{rec } y. x + y$           | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x \mid y$        | $\times$ |
| ► $\text{rec } x. \alpha. \text{rec } y. x \mid \beta. y$ | ✓        |



# Behavioral Equivalences

# Behavioral Equivalences



- ▶ How to compare the behavior of processes?
- ▶ Three different notions: graph isomorphism, trace equivalence, and bisimilarity.



- Syntactically different processes can exhibit exactly the same behavior.  
Examples:

|              |                    |                               |
|--------------|--------------------|-------------------------------|
| $\text{nil}$ | $\text{nil}[\phi]$ | $\text{nil} \setminus \alpha$ |
| $p$          | $p + \text{nil}$   | $p + p$                       |
| $p \mid q$   | $q \mid p$         |                               |
| $p + q$      | $q + p$            |                               |

- Syntactically different processes can exhibit exactly the same behavior.  
Examples:

|              |                    |                               |
|--------------|--------------------|-------------------------------|
| $\text{nil}$ | $\text{nil}[\phi]$ | $\text{nil} \setminus \alpha$ |
| $p$          | $p + \text{nil}$   | $p + p$                       |
| $p \mid q$   | $q \mid p$         | $p \mid \text{nil}$           |
| $p + q$      | $q + p$            |                               |

- In all cases, their LTS are different: the states are syntactically different.



- Syntactically different processes can exhibit exactly the same behavior.  
Examples:

|              |                    |                               |
|--------------|--------------------|-------------------------------|
| $\text{nil}$ | $\text{nil}[\phi]$ | $\text{nil} \setminus \alpha$ |
| $p$          | $p + \text{nil}$   | $p + p$                       |
| $p \mid q$   | $q \mid p$         | $p \mid \text{nil}$           |
| $p + q$      | $q + p$            |                               |

- In all cases, their LTS are different: the states are syntactically different.
- Using **graph isomorphism** we can abstract away from states.

# Graph Isomorphism



Two graphs  $G = (V, E)$  and  $G' = (V', E')$  are **isomorphic** if there exists a bijection  $f : V \rightarrow V'$  such that

$$v \xrightarrow{\mu} w \iff f(v) \xrightarrow{\mu} f(w)$$

# Graph Isomorphism



Two graphs  $G = (V, E)$  and  $G' = (V', E')$  are **isomorphic** if there exists a bijection  $f : V \rightarrow V'$  such that

$$v \xrightarrow{\mu} w \iff f(v) \xrightarrow{\mu} f(w)$$

## Example

Four intuitively equivalent processes.

$\alpha$   
 $\downarrow \alpha$   
 $\text{nil}$

$\alpha + \text{nil}$   
 $\downarrow \alpha$   
 $\text{nil}$

$\alpha + \alpha$   
 $\downarrow \alpha$   
 $\text{nil}$

$\alpha \mid \text{nil}$   
 $\downarrow \alpha$   
 $\text{nil} \mid \text{nil}$

# Graph Isomorphism

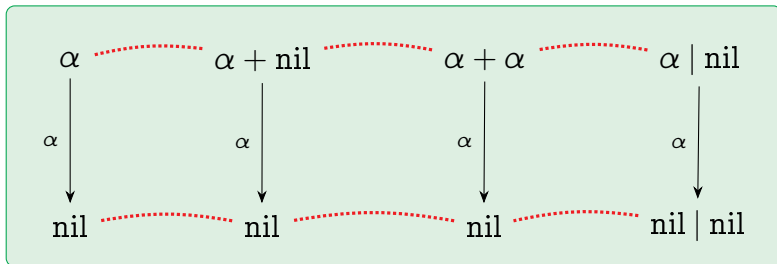


Two graphs  $G = (V, E)$  and  $G' = (V', E')$  are **isomorphic** if there exists a bijection  $f : V \rightarrow V'$  such that

$$v \xrightarrow{\mu} w \iff f(v) \xrightarrow{\mu} f(w)$$

## Example

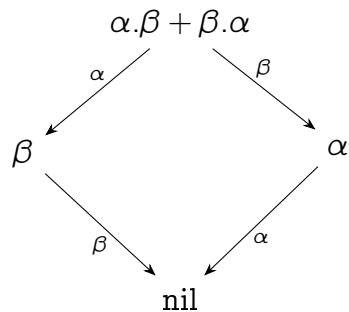
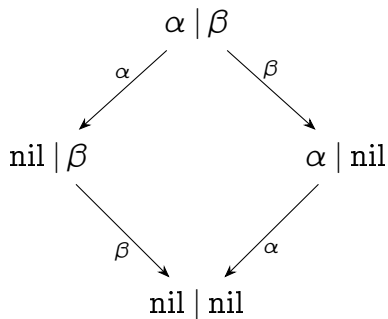
Four intuitively equivalent processes. Dotted lines indicate the isomorphism.



# Another Graph Isomorphism



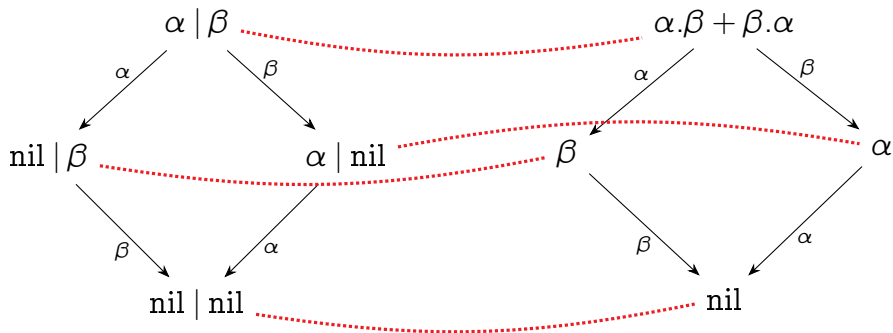
$$\alpha \mid \beta \quad \alpha.\beta + \beta.\alpha$$



# Another Graph Isomorphism



$$\alpha \mid \beta \quad \alpha.\beta + \beta.\alpha$$

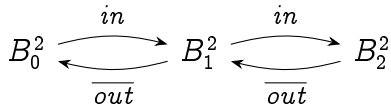


# Equivalent but Not Isomorphic



Consider again the buffer of capacity two.

- The buffer's intended **specification**:

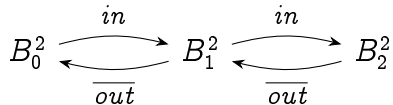


# Equivalent but Not Isomorphic

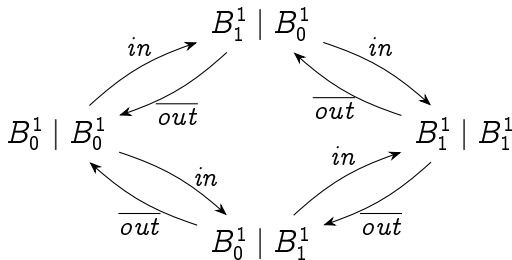


Consider again the buffer of capacity two.

- The buffer's intended **specification**:



- A parallel **implementation**:

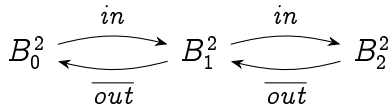


# Equivalent but Not Isomorphic

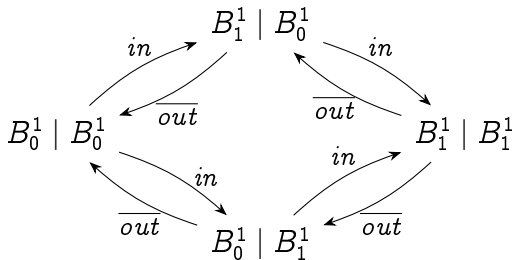


Consider again the buffer of capacity two.

- The buffer's intended **specification**:



- A parallel **implementation**:



- Morally, state  $B_1^2$  and is implemented by both  $B_1^1 | B_0^1$  and  $B_0^1 | B_1^1$ .

# Isomorphism is Too Strict



- ▶ Two processes with isomorphic LTS must be considered as equivalent.
- ▶ Graph isomorphism captures some interesting equivalences, but is it enough?
- ▶ Another example of intuitively equivalent, but not isomorphic processes:

$$P \triangleq \alpha.P \quad Q \triangleq \alpha.\alpha.Q \quad \alpha.P$$

# Trace Equivalence



- ▶ In automata theory: language equivalence.
- ▶ A (finite) **trace**  $p \xrightarrow{\mu_1 \mu_2 \dots \mu_k} q$  where

$$p = p_0 \xrightarrow{\mu_1} p_1 \xrightarrow{\mu_2} \dots \xrightarrow{\mu_k} p_k = q$$

# Trace Equivalence



- ▶ In automata theory: language equivalence.
- ▶ A (finite) **trace**  $p \xrightarrow{\mu_1 \mu_2 \cdots \mu_k} q$  where

$$p = p_0 \xrightarrow{\mu_1} p_1 \xrightarrow{\mu_2} \cdots \xrightarrow{\mu_k} p_k = q$$

- ▶ The **trace semantics** of a process

$$\mathcal{T}(p) = \{\mu_1 \mu_2 \cdots \mu_k \mid \exists q. p \xrightarrow{\mu_1 \mu_2 \cdots \mu_k} q\}$$

# Trace Equivalence



- ▶ In automata theory: language equivalence.
- ▶ A (finite) **trace**  $p \xrightarrow{\mu_1 \mu_2 \cdots \mu_k} q$  where

$$p = p_0 \xrightarrow{\mu_1} p_1 \xrightarrow{\mu_2} \cdots \xrightarrow{\mu_k} p_k = q$$

- ▶ The **trace semantics** of a process

$$\mathcal{T}(p) = \{\mu_1 \mu_2 \cdots \mu_k \mid \exists q. p \xrightarrow{\mu_1 \mu_2 \cdots \mu_k} q\}$$

- ▶ **Trace equivalence**

Two processes are equivalent if they have the same trace semantics:

$$p \equiv_{\text{tr}} q \iff \mathcal{T}(p) = \mathcal{T}(q)$$

# Trace Equivalence



## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:



## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:

►  $\mathcal{T}(P) = \{\overline{coin} \ coffee\}$



## Example

Given  $P \triangleq \overline{coin}.coffee.nil$  and  $M \triangleq coin.(\overline{coffee}.nil + \overline{tea}.nil)$ , we have:

- ▶  $\mathcal{T}(P) = \{\overline{coin} \ coffee\}$
- ▶  $\mathcal{T}(M) = \{coin \ \overline{coffee}, coin \ \overline{tea}\}$

# Trace Equivalence



- ▶ Graph isomorphism implies trace equivalence
- ▶ Trace equivalence preserves all the useful equivalences seen before

$$p \equiv_{\text{tr}} p + \text{nil} \equiv_{\text{tr}} p + p \equiv_{\text{tr}} p \mid \text{nil}$$

$$p + q \equiv_{\text{tr}} q + p$$

$$p \mid q \equiv_{\text{tr}} q \mid p$$

- ▶ Trace semantics is **prefix-closed**: if a trace is present, all its prefixes are also present
- ▶ Let's examine examples of trace-equivalent processes that are not graph isomorphic

# Trace Equivalence



$$P \triangleq \alpha.P \quad Q \triangleq \alpha.\alpha.Q \quad \alpha.P$$

# Trace Equivalence



$$P \triangleq \alpha.P \quad Q \triangleq \alpha.\alpha.Q \quad \alpha.P$$

►  $\mathcal{T}(\text{rec } x. \alpha.x) = \{\alpha^n \mid n \in \mathbb{N}\}$

# Trace Equivalence



$$P \triangleq \alpha.P \quad Q \triangleq \alpha.\alpha.Q \quad \alpha.P$$

- ▶  $\mathcal{T}(\text{rec } x. \alpha.x) = \{\alpha^n \mid n \in \mathbb{N}\}$
- ▶  $\mathcal{T}(\text{rec } x. \alpha.\alpha.x) = \{\alpha^n \mid n \in \mathbb{N}\}$

# Trace Equivalence



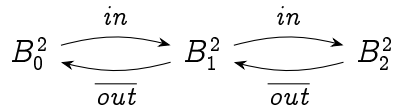
$$P \triangleq \alpha.P \qquad Q \triangleq \alpha.\alpha.Q \qquad \alpha.P$$

- ▶  $\mathcal{T}(\text{rec } x. \alpha.x) = \{\alpha^n \mid n \in \mathbb{N}\}$
- ▶  $\mathcal{T}(\text{rec } x. \alpha.\alpha.x) = \{\alpha^n \mid n \in \mathbb{N}\}$
- ▶  $\mathcal{T}(\alpha.\text{rec } x. \alpha.x) = \{\alpha^n \mid n \in \mathbb{N}\}$

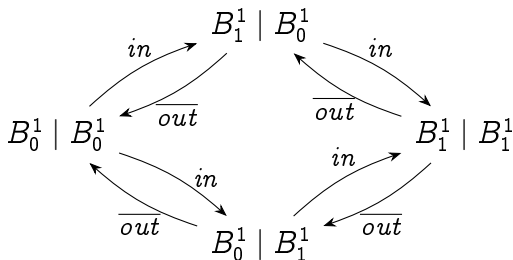
# Trace Equivalence



- Once again, the buffer's intended specification:



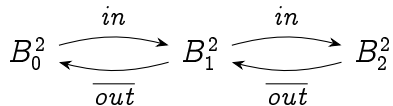
- A parallel implementation:



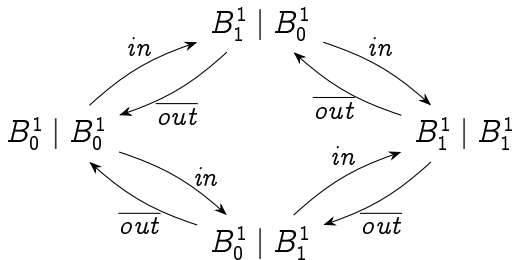
# Trace Equivalence



- Once again, the buffer's intended specification:



- A parallel implementation:



- $\mathcal{T}(B_0^2) = \mathcal{T}(B_0^1 | B_0^1)$ .

Intuition: For any prefix of a trace, we have  $0 \leq \#in - \#out \leq 2$ .

# Two Vending Machines

(Not isomorphic, but trace equivalent)

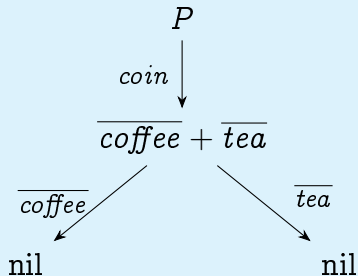
$$P \triangleq \text{coin} . (\overline{\text{coffee}} + \overline{\text{tea}})$$

$$Q \triangleq \text{coin} . \overline{\text{coffee}} + \text{coin} . \overline{\text{tea}}$$

# Two Vending Machines

(Not isomorphic, but trace equivalent)

$$P \triangleq \text{coin} . (\overline{\text{coffee}} + \overline{\text{tea}})$$

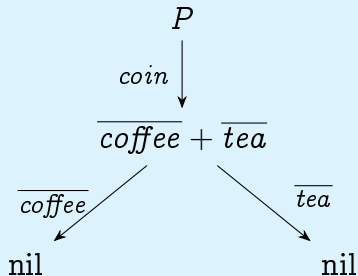


$$Q \triangleq \text{coin} . \overline{\text{coffee}} + \text{coin} . \overline{\text{tea}}$$

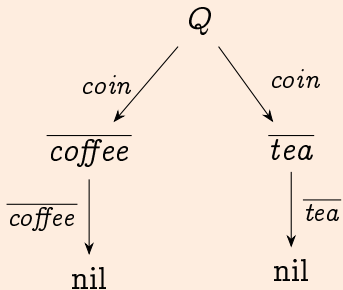
# Two Vending Machines

(Not isomorphic, but trace equivalent)

$$P \triangleq \text{coin} . (\overline{\text{coffee}} + \overline{\text{tea}})$$



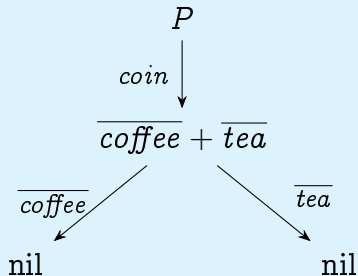
$$Q \triangleq \text{coin} . \overline{\text{coffee}} + \text{coin} . \overline{\text{tea}}$$



# Two Vending Machines

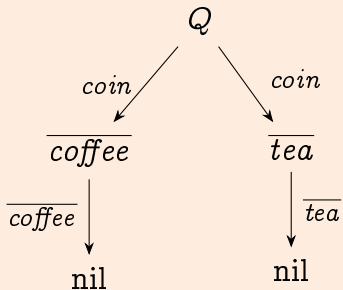
(Not isomorphic, but trace equivalent)

$$P \triangleq \text{coin} . (\overline{\text{coffee}} + \overline{\text{tea}})$$



As a user, you insert a coin, then choose the drink

$$Q \triangleq \text{coin} . \overline{\text{coffee}} + \text{coin} . \overline{\text{tea}}$$

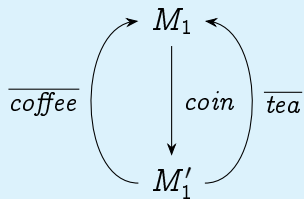


As a user, you insert a coin, then get the drink chosen by the machine

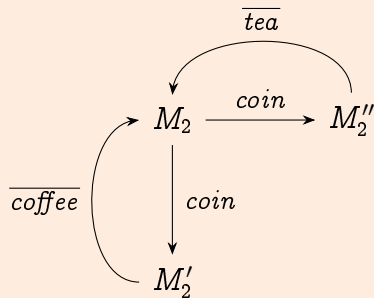
# Recursive Machines



$$M_1 \triangleq \text{coin} . \underbrace{(\overline{\text{coffee}} . M_1 + \overline{\text{tea}} . M_1)}_{M'_1}$$

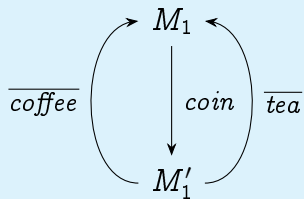


$$M_2 \triangleq \text{coin} . \underbrace{\overline{\text{coffee}} . M_2}_{M'_2} + \text{coin} . \underbrace{\overline{\text{tea}} . M_2}_{M''_2}$$



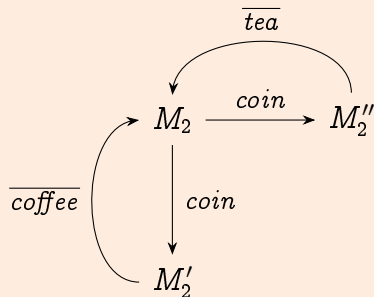


$$M_1 \triangleq \text{coin} . \underbrace{(\overline{\text{coffee}} . M_1 + \overline{\text{tea}} . M_1)}_{M'_1}$$



We have  $\mathcal{T}(M_1) = \mathcal{T}(M_2)$ .

$$M_2 \triangleq \text{coin} . \underbrace{\overline{\text{coffee}} . M_2}_{M'_2} + \text{coin} . \underbrace{\overline{\text{tea}} . M_2}_{M''_2}$$



# A System of Recursive Machines



$$M_1 \triangleq \text{coin} . \underbrace{(\overline{\text{coffee}} . M_1 + \overline{\text{tea}} . M_1)}_{M'_1}$$

$$M_2 \triangleq \text{coin} . \underbrace{\overline{\text{coffee}} . M_2}_{M'_2} + \text{coin} . \underbrace{\overline{\text{tea}} . M_2}_{M''_2}$$

Now define  $C \triangleq \overline{\text{coin}} . \underbrace{\text{coffee} . C}_{C'}$  and  $S \triangleq \{\text{coin}, \text{coffee}, \text{tea}\}$ .

# A System of Recursive Machines

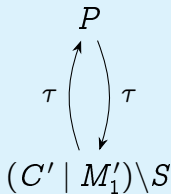


$$M_1 \triangleq \text{coin} . \underbrace{(\overline{\text{coffee}} . M_1 + \overline{\text{tea}} . M_1)}_{M'_1}$$

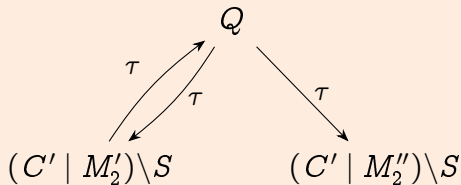
$$M_2 \triangleq \text{coin} . \underbrace{\overline{\text{coffee}} . M_2}_{M'_2} + \text{coin} . \underbrace{\overline{\text{tea}} . M_2}_{M''_2}$$

Now define  $C \triangleq \overline{\text{coin}} . \underbrace{\text{coffee} . C}_{C'}$  and  $S \triangleq \{\text{coin}, \text{coffee}, \text{tea}\}$ . Two systems:

$$P \triangleq (C \mid M_1) \setminus S$$



$$Q \triangleq (C \mid M_2) \setminus S$$



# A System of Recursive Machines

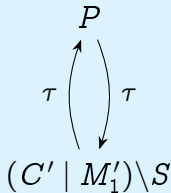


$$M_1 \triangleq \text{coin} . \underbrace{(\overline{\text{coffee}} . M_1 + \overline{\text{tea}} . M_1)}_{M'_1}$$

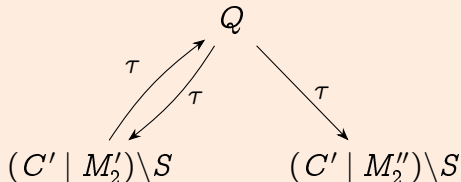
$$M_2 \triangleq \text{coin} . \underbrace{\overline{\text{coffee}} . M_2}_{M'_2} + \text{coin} . \underbrace{\overline{\text{tea}} . M_2}_{M''_2}$$

Now define  $C \triangleq \overline{\text{coin}} . \underbrace{\text{coffee} . C}_{C'}$  and  $S \triangleq \{\text{coin}, \text{coffee}, \text{tea}\}$ . Two systems:

$$P \triangleq (C \mid M_1) \setminus S$$



$$Q \triangleq (C \mid M_2) \setminus S$$



We have  $\mathcal{T}(P) = \mathcal{T}(Q)$ .

# A System of Recursive Machines

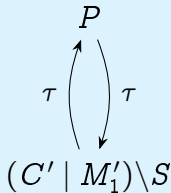


$$M_1 \triangleq \text{coin} . \underbrace{(\overline{\text{coffee}} . M_1 + \overline{\text{tea}} . M_1)}_{M'_1}$$

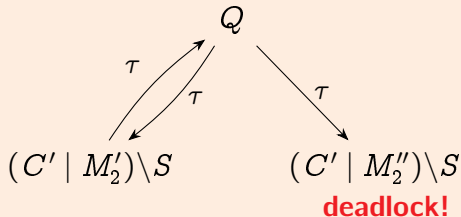
$$M_2 \triangleq \text{coin} . \underbrace{\overline{\text{coffee}} . M_2}_{M'_2} + \text{coin} . \underbrace{\overline{\text{tea}} . M_2}_{M''_2}$$

Now define  $C \triangleq \overline{\text{coin}} . \underbrace{\text{coffee} . C}_{C'}$  and  $S \triangleq \{\text{coin}, \text{coffee}, \text{tea}\}$ . Two systems:

$$P \triangleq (C \mid M_1) \setminus S$$



$$Q \triangleq (C \mid M_2) \setminus S$$



We have  $\mathcal{T}(P) = \mathcal{T}(Q)$ .



- ▶ Graph isomorphism distinguishes too many processes, but trace equivalence identifies too many.
- ▶ We need some notion of equivalence in between the two.
- ▶ We now introduce **strong bisimilarity**. Intuition: Two processes are equivalent unless we have some good reasons to distinguish them.

# A Bisimulation Game



Two processes,  $p$  and  $q$ , and two opposing players:

- ▶ **Alice**, the attacker, aims to prove that  $p$  and  $q$  are **not** equivalent.
- ▶ **Bob**, the defender, aims to prove  $p$  and  $q$  are equivalent.

# A Bisimulation Game



Two processes,  $p$  and  $q$ , and two opposing players:

- ▶ **Alice**, the attacker, aims to prove that  $p$  and  $q$  are **not** equivalent.
- ▶ **Bob**, the defender, aims to prove  $p$  and  $q$  are equivalent.

The game is turn-based. At each turn:

- ▶ **Alice** chooses one process and one of its outgoing transitions.
- ▶ **Bob** must reply with a transition of the other process, matching the label of the transition chosen by Alice.

# A Bisimulation Game



Two processes,  $p$  and  $q$ , and two opposing players:

- ▶ **Alice**, the attacker, aims to prove that  $p$  and  $q$  are **not** equivalent.
- ▶ **Bob**, the defender, aims to prove  $p$  and  $q$  are equivalent.

The game is turn-based. At each turn:

- ▶ **Alice** chooses one process and one of its outgoing transitions.
- ▶ **Bob** must reply with a transition of the other process, matching the label of the transition chosen by Alice.

At the next turn, if any, the players will consider the equivalence of the target processes of the chosen transitions.

# A Bisimulation Game



**Alice wins** if, at some point, she can make a move that Bob cannot match

**Bob wins** in all other cases:  
if Alice cannot find a move  
if the game does not terminate

# A Bisimulation Game



**Alice wins** if, at some point, she can make a move that Bob cannot match

**Bob wins** in all other cases:  
if Alice cannot find a move  
if the game does not terminate

Alice has a **winning strategy**

- ▶ if she can make a move that Bob cannot match;
- ▶ or if she can make a move that no matter what Bob replies, at the next turn she wins;
- ▶ or so the like after any (finite) number of moves...

In other words, Alice has a winning strategy if she can disprove the equivalence of  $p$  and  $q$  in a finite number of moves.

# Example: The Vending Machines



$coin.(\overline{coffee} + \overline{tea})$

$\downarrow coin$

$\overline{coffee} + \overline{tea}$

$\swarrow \overline{coffee}$

$nil$

$\searrow \overline{tea}$

$nil$

$coin.\overline{coffee} + coin.\overline{tea}$

$\swarrow coin$

$\overline{coffee}$

$\downarrow \overline{coffee}$

$nil$

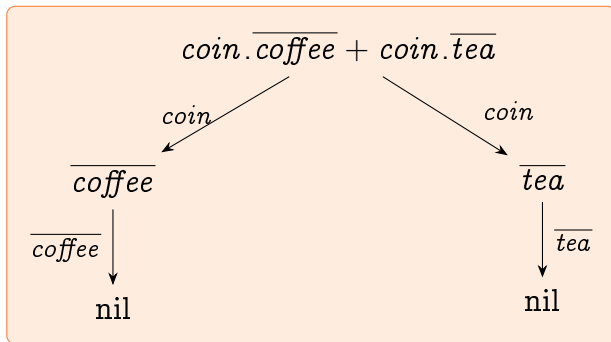
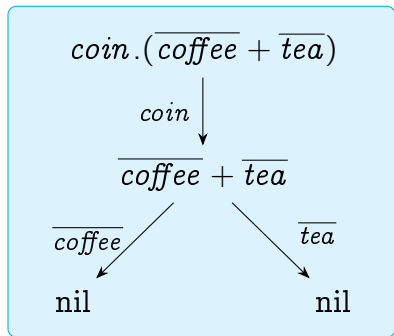
$\searrow coin$

$\overline{tea}$

$\downarrow \overline{tea}$

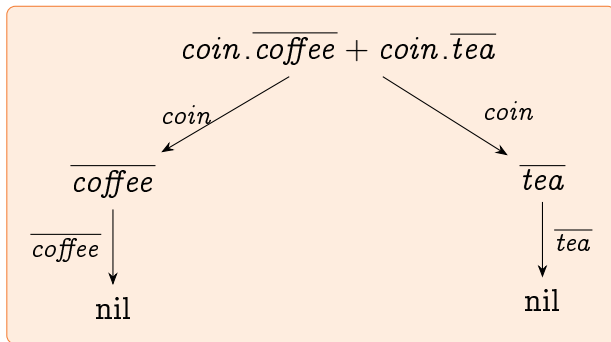
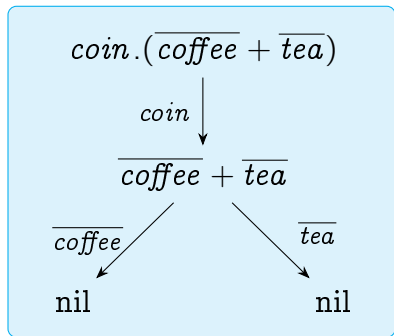
$nil$

# Example: The Vending Machines



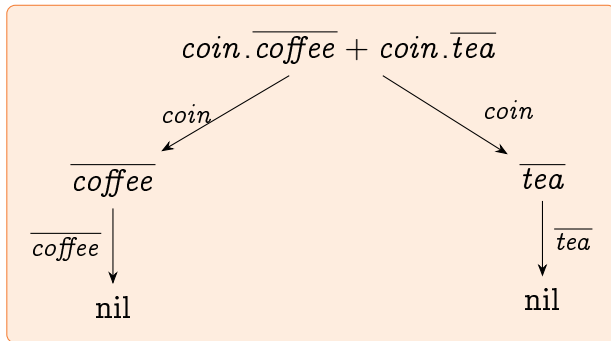
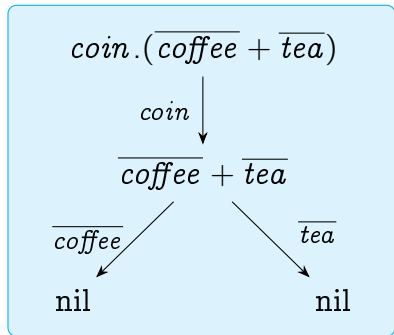
► **Alice** plays RHS:  $\text{coin.coffee} + \text{coin.tea} \xrightarrow{\text{coin}} \text{coffee}$

# Example: The Vending Machines



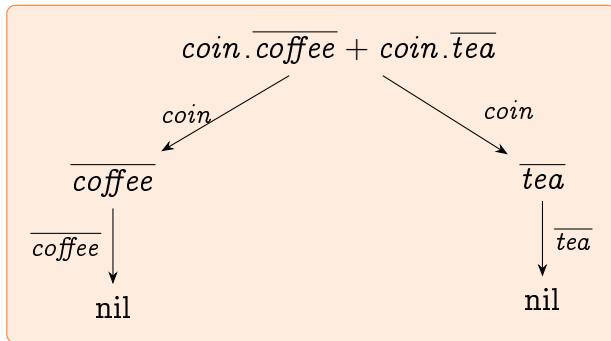
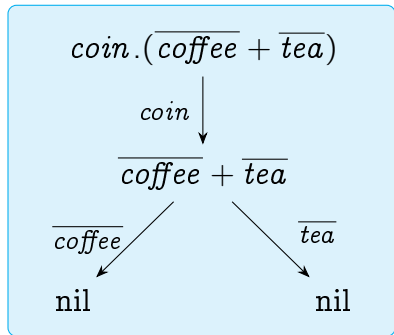
- ▶ **Alice** plays RHS:  $coin.\overline{coffee} + coin.\overline{tea} \xrightarrow{coin} \overline{coffee}$
- ▶ **Bob** can only reply (LHS):  $coin.(\overline{coffee} + \overline{tea}) \xrightarrow{coin} \overline{coffee}$

# Example: The Vending Machines



- ▶ **Alice** plays RHS:  $\text{coin} . \overline{\text{coffee}} + \text{coin} . \overline{\text{tea}} \xrightarrow{\text{coin}} \overline{\text{coffee}}$
- ▶ **Bob** can only reply (LHS):  $\text{coin} . (\overline{\text{coffee}} + \overline{\text{tea}}) \xrightarrow{\text{coin}} \overline{\text{coffee}}$
- ▶ Now **Alice** plays LHS:  $\overline{\text{coffee}} + \overline{\text{tea}} \xrightarrow{\text{tea}} \text{nil}$

# Example: The Vending Machines



- ▶ **Alice** plays RHS:  $\text{coin} . \overline{\text{coffee}} + \text{coin} . \overline{\text{tea}} \xrightarrow{\text{coin}} \overline{\text{coffee}}$
- ▶ **Bob** can only reply (LHS):  $\text{coin} . (\overline{\text{coffee}} + \overline{\text{tea}}) \xrightarrow{\text{coin}} \overline{\text{coffee}}$
- ▶ Now **Alice** plays LHS:  $\overline{\text{coffee}} + \overline{\text{tea}} \xrightarrow{\text{tea}} \text{nil}$
- ▶ But **Bob** cannot reply:  $\overline{\text{coffee}} \not\xrightarrow{\text{tea}}$ .

**Alice wins!**



- ▶ **Bisimulation** is a general notion in Computer Science: it applies to any LTS, not just to CCS processes.
- ▶ We study Milner's original definition of **strong bisimulation relation**.
- ▶ There are many strong bisimulations, but we are interested in the largest such relation, called **strong bisimilarity**.

## The Bisimulation Proof Method

To prove that two processes are *strongly bisimilar* it is enough to show they are related by a *strong bisimulation*.

# Strong Bisimulation



Consider

- ▶ A set of processes  $\mathcal{P}$ .
- ▶ A binary relation  $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ . We write  $p \mathcal{R} q$  when  $(p, q) \in \mathcal{R}$

$\mathcal{R}$  is a **strong bisimulation** if and only if

$$\forall p, q. (p, q) \in \mathcal{R} \Rightarrow \begin{cases} \forall \mu, p'. p \xrightarrow{\mu} p' \Rightarrow \exists q'. q \xrightarrow{\mu} q' \wedge p' \mathcal{R} q' \\ \wedge \\ \forall \mu, q'. q \xrightarrow{\mu} q' \Rightarrow \exists p'. p \xrightarrow{\mu} p' \wedge p' \mathcal{R} q' \end{cases}$$

# Strong Bisimulation



Consider

- ▶ A set of processes  $\mathcal{P}$ .
- ▶ A binary relation  $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ . We write  $p \mathcal{R} q$  when  $(p, q) \in \mathcal{R}$

$\mathcal{R}$  is a **strong bisimulation** if and only if

$$\forall p, q. (p, q) \in \mathcal{R} \Rightarrow \left\{ \begin{array}{l} \forall \mu, p'. p \xrightarrow{\mu} p' \Rightarrow \exists q'. q \xrightarrow{\mu} q' \wedge p' \mathcal{R} q' \\ \wedge \\ \forall \mu, q'. q \xrightarrow{\mu} q' \Rightarrow \exists p'. p \xrightarrow{\mu} p' \wedge p' \mathcal{R} q' \end{array} \right.$$

**Intuition:** If two processes are related, then for any move of Alice, Bob can find a move that leads to related processes. That is, Bob has a *winning strategy*.



- The empty relation and the identity relation

$$\emptyset \quad Id \triangleq \{(p, p) \mid p \in \mathcal{P}\}$$

are both strong bisimulations.

- Any graph isomorphism defines a strong bisimulation

$$\mathcal{R}_f \triangleq \{(p, f(p))\}$$

# Example

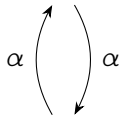


Are these processes bisimilar?

$\text{rec } x. \alpha.x$



$\text{rec } x. \alpha.\alpha.x$

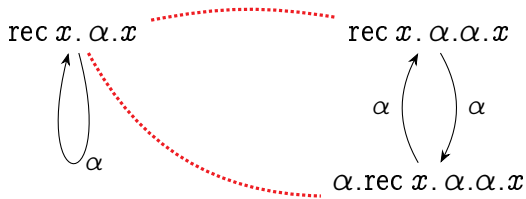


$\alpha.\text{rec } x. \alpha.\alpha.x$

# Example



Are these processes bisimilar?



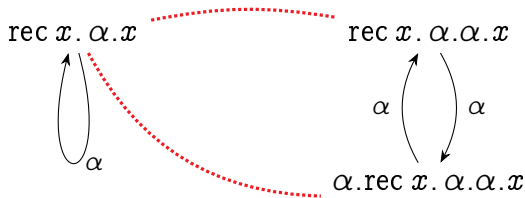
Yes, we have a strong bisimulation relation:

$$\mathcal{R} \triangleq \{(\text{rec } x. \alpha.x, \text{rec } x. \alpha.\alpha.x), \\ (\text{rec } x. \alpha.x, \alpha.\text{rec } x. \alpha.\alpha.x)\}$$

# Example



Are these processes bisimilar?

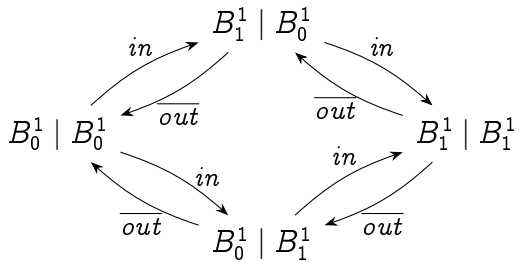
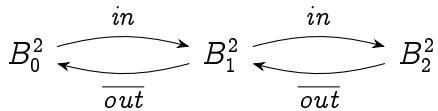


Yes, we have a strong bisimulation relation:

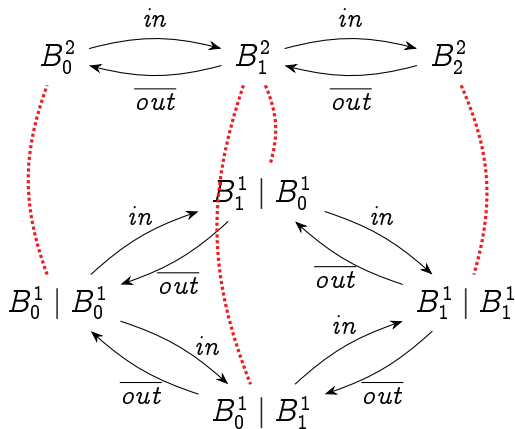
$$\mathcal{R} \triangleq \{(\text{rec } x. \alpha.x, \text{rec } x. \alpha.\alpha.x), \\ (\text{rec } x. \alpha.x, \alpha.\text{rec } x. \alpha.\alpha.x)\}$$

**Note:** The same process can be related to many processes!

# Example



# Example



We have the strong bisimulation:

$$\mathcal{R} \triangleq \{(B_0^2, B_0^1 | B_0^1), \\ (B_1^2, B_0^1 | B_1^1), \\ (B_1^2, B_1^1 | B_0^1), \\ (B_2^2, B_1^1 | B_1^1)\}$$

# Summing Up



Today:

- ▶ Linked Buffers, revisited
- ▶ Graph isomorphisms, trace equivalence, strong bisimilarity

Next:

Properties of strong bisimilarity and weak bisimilarity.



The End